

Solution d'Examen de rattrapage Système d'Exploitation II

Exercice 1 : (6 pts)

1) L'état est sûr. Car : (2 pts)

- 1- Demande de P3 $\leq$ D, Marquer P3, D = D + Allocation de P3, D = (6, 4, 2, 2)
- 2- Demande de P1 $\leq$ D, Marquer P1, D = D + Allocation de P1, D = (7, 4, 2, 3)
- 3- Demande de P4 $\leq$ D, Marquer P4, D = D + Allocation de P4, D = (7, 5, 2, 6)
- 4- Demande de P2 $\leq$ D, Marquer P2, D = D + Allocation de P2, D = (8, 7, 2, 6)

2) Le nouvel état sera :

D : Demandes				
	R1	R2	R3	R4
P1	3	2	0	2
P2	2	5	3	0
P3	4	0	0	1
P4	1	0	1	0

A : Allocations				
	R1	R2	R3	R4
P1	1	0	0	1
P2	1	1	0	0
P3	1	1	0	1
P4	0	1	0	3

D : Disponibles			
R1	R2	R3	R4
5	4	2	1

2.1) (2 pts)

- 1- Demande de P3 $\leq$ D, Marquer P3, D = D + Allocation de P3, D = (6, 5, 2, 2)
 - 2- Demande de P1 $\leq$ D, Marquer P1, D = D + Allocation de P1, D = (7, 5, 2, 3)
 - 3- Demande de P4 $\leq$ D, Marquer P4, D = D + Allocation de P4, D = (7, 6, 2, 6)
 - 4- Demande de P2 $>$ D alors P2 ne sera pas marqué et le système se trouve dans un état de DEADLOCK.
- Le nouvel état n'est pas un état sûr.

Exercice 2 : (8 pts)

1) Solution avec les sémaphores (4 pts)

```
const int N = ...;           // nombre total d'étudiants
semaphore mutex = 1;        // protection accès variable compteur
semaphore db = 1;           // accès exclusif à la base de données
semaphore juryPret = 0;     // signal que tous les dépôts sont faits
int compteur = 0;           // nombre d'étudiants ayant déposé
```

```
process Etudiant(int i){
    wait(db);                // accès exclusif à la base
    deposer_projet(i);       // dépose son projet
    signal(db);
    wait(mutex);
    compteur++;
    if (compteur == N) {
        signal(juryPret);   // dernier étudiant réveille le jury
    }
    signal(mutex);
}
```

```

process Jury(){
    wait(juryPret);    // attend que tous les étudiants aient déposé
    wait(db);          // accès exclusif à la base (bloque les étudiants)
    lire_et_evaluer_projets();
    signal(db);
}

```

2) Solution avec le moniteur (4 pts)

```

monitor DepotProjets {
    const int N;          // Nombre total d'étudiants
    int compteur = 0;
    condition juryPret;
    condition dbLibre;
    bool dbOccupeParJury = false;

    void deposerProjet(int i) {
        if dbOccupeParJury then cwait(dbLibre); // Attend que le jury fini
        deposer_projet(i);    // Dépôt du projet
        compteur ++;
        if compteur = N then csignal(juryPret);
    }

    void evaluerProjets() {
        if compteur < N then cwait(juryPret); // Attend que tous les étudiants déposent leur projets
        dbOccupeParJury = true;
        lire_et_evaluer_projets(); // Lecture/évaluation des projets
        dbOccupeParJury = false;
        csignal(dbLibre);          // Réveille un étudiant en attente
    }
}

processus Etudiant(i) {
    DepotProjets.deposerProjet(i);
}
processus Jury {
    DepotProjets.evaluerProjets();
}

```

Exercice 3 : (6 pts)

```
#include <pthread.h>
```

```
void* somme(void* arg){  
    int X=0, Y=0, res=0;  
    printf("Donner la valeur de X:");  
    scanf("%d",&X);  
    printf("Donner la valeur de Y:");  
    scanf("%d",&Y);  
    res1 = X + Y;  
    return (void*) res ;  
}
```

```
void* division(void* arg){  
    int* t = (int*)arg;  
    int a = t[0];  
    int b = t[1];  
    int c = a / b;  
    return (void*) c ;  
}
```

```
int main(){  
    pthread_t t1, t2;  
    int tab[2];  
    int Z;  
    void* resSum, resDiv;  
    pthread_create(&t1,NULL,&somme,NULL);  
    pthread_join(t1,&resSum);  
    tab[0] = (int) resSum;  
  
    printf("Donner la valeur de Z:");  
    scanf("%d",&Z);  
    tab[1] = Z ;  
    pthread_create(&t2,NULL,&division,(void*)&tab);  
    pthread_join(t2,&resD);  
    printf("Le resultat de F = %d",(int) resD);  
    system("PAUSE");  
    return 0;  
}
```